

中山大学计算机院本科生实验报告

(2024年春季学期)

课程名称：并行程序设计

批改人：

实验	10-CUDA卷积	专业	计算机科学与技术
学号	22336203	姓名	沈苇杭
Email	shenwh7@mail2.sysu.edu.cn	完成日期	2025.6.11

实验目的

- 掌握基本的CUDA并行API。
- 掌握直接卷积、im2col卷积和cuDNN卷积的方法
- 分析图像大小、访存方式、任务/数据划分方式、线程块大小等因素对卷积程序性能的影响。

实验过程与核心代码

直接卷积

```
image = Image.open('image.jpg')
image.save(fp='image.ppm', format='PPM')
```

想要对图像做卷积，需要将图像转化为数字形式。在主机上使用python PIL 将.jpg格式的图片转换为.ppm格式，再上传到实验平台。

```
bool read_ppm(const std::string& filename, std::vector<unsigned char>& image, int& width, int& height) {
    std::ifstream file(filename, std::ios::binary);
    if (!file) return false;

    std::string magic;
    file >> magic;
    if (magic != "P6") return false;

    file.ignore();
    file >> width >> height;
    int max_val;
    file >> max_val;
    file.ignore();

    image.resize(width * height * 3);
    file.read(reinterpret_cast<char*>(image.data()), image.size());
    return true;
}
```

在卷积程序中读取.ppm文件。输入图像名、图像容器、图像宽高的引用，按二进制读取图像数据并存储到图像容器中。返回一个布尔值表示读取成功与否。

```
float h_kernel[KERNEL_NUM * CHANNELS * KERNEL_SIZE * KERNEL_SIZE];
for (int i = 0; i < KERNEL_NUM * CHANNELS * KERNEL_SIZE * KERNEL_SIZE; ++i)
    h_kernel[i] = 1.0f;
```

读取.ppm文件后，计算图片的总像素数。对图像像素标准化，并全部放入一个总像素数大小的vector容器中。

初始化卷积核。使用一个float数组来存放所有的卷积核。卷积核的每个元素初始化为1，这样可以起到对图片做模糊处理的效果。

分配GPU内存。在GPU上分配两块内存，一块存储图像像素，一块存储卷积核。

将数据从主机内存传输到GPU内存。

```
for (int stride : {1, 2, 3}) {
    int padding = (KERNEL_SIZE - 1 - (stride - 1)) / 2 + 1;
    int outH = height - 2;
    int outW = width - 2;

    std::vector<float> h_output(KERNEL_NUM * outH * outW);
    cudaMalloc(&d_output, sizeof(float) * h_output.size());

    dim3 block(16, 16);
    dim3 grid((outW + 15) / 16, (outH + 15) / 16, KERNEL_NUM);

    cudaEvent_t start, stop;
    cudaEventCreate(&start);
    cudaEventCreate(&stop);
    cudaEventRecord(start);

    conv2d_kernel<<<grid, block>>>(d_input, d_kernel, d_output,
                                     height, width, outH, outW,
                                     stride, padding);

    cudaEventRecord(stop);
    cudaEventSynchronize(stop);
```

分别计算stride为1,2,3的结果。

计算不同stride时的padding和输出尺寸。

在主机上分配用于存储输出的内存，设置CUDA线程块和网格的大小，调用CUDA内核进行卷积运算，将结果从GPU复制到主机。并且，记录运行时间。

将输出数据按照RGB通道重新组织成输出图像，并保存为.ppm格式。

回收内存。

```
// cuda kernal
__global__ void conv2d_kernel(const float* input, const float* kernels, float* output,
                              int H, int W, int outH, int outW, int stride, int padding) {
    int out_x = blockIdx.x * blockDim.x + threadIdx.x;
    int out_y = blockIdx.y * blockDim.y + threadIdx.y;
    int k = blockIdx.z;
    if (out_x >= outW || out_y >= outH || k >= KERNEL_NUM) return;

    float sum = 0;
    for (int c = 0; c < CHANNELS; ++c) {
        for (int dy = 0; dy < KERNEL_SIZE; ++dy) {
            for (int dx = 0; dx < KERNEL_SIZE; ++dx) {
                int in_y = out_y * stride + dy - padding;
                int in_x = out_x * stride + dx - padding;
                float val = 0;
                if (in_y >= 0 && in_y < H && in_x >= 0 && in_x < W)
                    val = input[(c * H + in_y) * W + in_x];
                float kernel_val = kernels[(k * CHANNELS + c) * KERNEL_SIZE + dy] * KERNEL_SIZE + dx];
                sum += val * kernel_val;
            }
        }
    }
    output[(k * outH + out_y) * outW + out_x] = sum;
}
```

核函数对图像做直接卷积。输入图像数据的指针、卷积核数据的指针、输出数据的指针、输入和输出图像的高度宽度、stride和Padding。在函数中，首先计算出当前线程负责的输出像素位置。随后，对每个卷积核的每个元素，计算当前输出位置对应的输入位置，再计算输入值和卷积核元素的卷积结果。

im2col卷积

```
__global__ void im2col_kernel(const float* input, float* columns,
                             int C, int H, int W,
                             int KH, int KW,
                             int outH, int outW,
                             int stride, int padding) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    int total_cols = outH * outW;

    for (int i = 0; i < 4; ++i) {
        int col = tid * 4 + i;
        if (col >= total_cols) return;

        int out_y = col / outW;
        int out_x = col % outW;

        int col_offset = 0;
        for (int c = 0; c < C; ++c) {
            for (int ky = 0; ky < KH; ++ky) {
                for (int kx = 0; kx < KW; ++kx) {
                    int in_y = out_y * stride + ky - padding;
                    int in_x = out_x * stride + kx - padding;
                    float val = 0.0f;
                    if (in_y >= 0 && in_y < H && in_x >= 0 && in_x < W)
                        val = input[(c * H + in_y) * W + in_x];
                    columns[col * (C * KH * KW) + col_offset++] = val;
                }
            }
        }
    }
}
```

g)

```
void run_im2col_convolution(const float* d_input, const float* d_kernel,
                           float* d_output, float* d_columns,
                           int C, int H, int W,
                           int KH, int KW, int K,
                           int stride, int padding,
                           int outH, int outW) {
    int col_size = C * KH * KW;
    int num_output = outH * outW;

    dim3 block(1024);
    dim3 grid((num_output + block.x - 1) / block.x);
    size_t shared_mem_size = sizeof(float) * C * KH * KW;
    im2col_kernel<<<grid, block>>>(d_input, d_columns, C, H, W, KH, KW, outH, outW, stride, padding);
    //im2col_shared_kernel<<<grid, block, shared_mem_size>>>(d_input, d_columns, C, H, W, KH, KW, outH, outW, stride, padding);

    cublasHandle_t handle;
    cublasCreate(&handle);
    float alpha = 1.0f, beta = 0.0f;

    cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N,
                K, num_output, col_size,
                &alpha,
                d_kernel, K,
                d_columns, col_size,
                &beta,
                d_output, K);

    cublasDestroy(handle);
}
```

资源 编译日志 调试 搜索结果

ng)

核函数对图像做im2col卷积。将图像转换为列矩阵，然后使用矩阵乘法来计算卷积。

cuDNN卷积

使用cuDNN卷积时，需要首先创建cuDNN描述符并定义卷积操作的参数，随后调用cuDNN提供的 `cudaConvolutionForward` 函数执行卷积，并记录运算时间。

```
cudaTensorDescriptor_t input_desc, output_desc;
cudaFilterDescriptor_t filter_desc;
cudaConvolutionDescriptor_t conv_desc;
cudaCreateTensorDescriptor(&input_desc);
cudaCreateFilterDescriptor(&filter_desc);
cudaCreateConvolutionDescriptor(&conv_desc);
int pad = (KERNEL_SIZE - 1) / 2;
cudaSetTensor4dDescriptor(input_desc, CUDNN_TENSOR_NCHW, CUDNN_DATA_FLOAT,
                           1, CHANNELS, height, width);
cudaSetFilter4dDescriptor(filter_desc, CUDNN_DATA_FLOAT, CUDNN_TENSOR_NCHW,
                           KERNEL_NUM, CHANNELS, KERNEL_SIZE, KERNEL_SIZE);
cudaSetConvolution2dDescriptor(conv_desc, pad, pad, stride, stride,
                               1, 1, CUDNN_CROSS_CORRELATION, CUDNN_DATA_FLOAT);
```

定义张量描述符、卷积核描述符、卷积操作描述符的变量，并创建描述符对象。

设置输入变量描述符。设置输入张量格式为NCHW格式，数据类型为float。

设置卷积核描述符。设置卷积核格式为NCHW格式，数据格式为float。

设置卷积操作描述符。设置卷积操作为普通二维卷积，有padding和stride。

```
int n, c, h, w;
cudaGetConvolution2dForwardOutputDim(conv_desc, input_desc, filter_desc, &n, &c, &h, &w);
cudaCreateTensorDescriptor(&output_desc);
cudaSetTensor4dDescriptor(output_desc, CUDNN_TENSOR_NCHW, CUDNN_DATA_FLOAT, n, c, h, w);
size_t workspace_size = 0;
cudaConvolutionFwdAlgo_t algo;
cudaGetConvolutionForwardAlgorithm(cuda, input_desc, filter_desc, conv_desc, output_desc,
                                   CUDNN_CONVOLUTION_FWD_PREFER_FASTEST, 0, &algo);
cudaGetConvolutionForwardWorkspaceSize(cuda, input_desc, filter_desc, conv_desc,
                                       output_desc, algo, &workspace_size);
```

使用 `cudaGetConvolution2dForwardOutputDim` 函数推导输出张量的维度，创建输出描述符对象。

设置输出张量描述符，格式与输入类似。

获得cuDNN推荐的卷积算法，获得workspace大小。

```
float alpha = 1, beta = 0;
cudaEvent_t start, stop;
cudaEventCreate(&start);
cudaEventCreate(&stop);
cudaEventRecord(start);
cudaConvolutionForward(cuda, &alpha,
                       input_desc, d_input,
                       filter_desc, d_kernel,
                       conv_desc, algo,
                       d_workspace, workspace_size,
                       &beta,
                       output_desc, d_output);
cudaEventRecord(stop);
cudaEventSynchronize(stop);
float ms;
cudaEventElapsedTime(&ms, start, stop);
std::cout << "Stride = " << stride << ", Time = " << ms << " ms\n";
```

调用 `cudaConvolutionForward` 计算卷积。 `cudaConvolutionForward` 函数的解释如下：

```
cudaStatus_t cudaConvolutionForward(  
    cudaHandle_t handle, // cuDNN上下文句柄  
    const void *alpha,    // 输入张量的缩放系数  
    const cudaTensorDescriptor_t xDesc, //输入张量的描述符  
    const void *x, // 输入张量的GPU地址  
    const cudaFilterDescriptor_t wDesc, // 卷积核的描述符  
    const void *w, //卷积核的GPU地址  
    const cudaConvolutionDescriptor_t convDesc, // 卷积操作的参数的描述符  
    cudaConvolutionFwdAlgo_t algo, // 卷积使用的算法  
    void *workspace, // 需要的GPU空间地址  
    size_t workspaceSizeInBytes, // GPU空间大小  
    const void *beta, //输出向量的缩放系数  
    const cudaTensorDescriptor_t yDesc, // 输出张量的描述符  
    void *y // 输出张量的GPU地址  
);
```

实验结果

直接卷积



上方是原图，下方是卷积处理后的图片。可以看出，这个全1的卷积核对图片做了模糊操作。

基础版本核函数

图像大小\线程块大小	8*8	16*16	32*32
176 * 193	stride 1 time = 0.09ms stride 2 time = 0.07ms stride 3 time = 0.06ms	stride 1 time = 0.20ms stride 2 time = 0.17ms stride 3 time = 0.16ms	stride 1 time = 0.21ms stride 2 time = 0.18ms stride 3 time = 0.17ms
472 * 524	stride 1 time = 1.27ms stride 2 time = 1.18ms stride 3 time = 1.12ms	stride 1 time = 1.26ms stride 2 time = 1.10ms stride 3 time = 1.06ms	stride 1 time = 1.27ms stride 2 time = 1.16ms stride 3 time = 1.11ms

从表中可以看出，线程块大小相同时，图片越大，运行时间越长。
图片大小相同时，线程块过大或过小，都会影响程序性能，但线程块大小对性能的影响不大。

修改任务划分方式（每个线程处理两个像素）

图像大小\线程块大小	8*8	16*16	32*32
176 * 193	stride 1 time = 0.21ms stride 2 time = 0.20ms stride 3 time = 0.19ms	stride 1 time = 0.67ms stride 2 time =0.57ms stride 3 time = 0.56ms	stride 1 time = 0.69ms stride 2 time =0.60ms stride 3 time = 0.60ms
472 * 524	stride 1 time = 1.40ms stride 2 time = 1.58ms stride 3 time = 1.30ms	stride 1 time = 4.91ms stride 2 time = 4.29ms stride 3 time = 4.14ms	stride 1 time = 4.80ms stride 2 time = 4.35ms stride 3 time = 4.19ms

从表中可以看出，增加每个线程处理的像素数量，会降低程序性能。

共享内存

图像大小\线程块大小	8*8	16*16	32*32
176 * 193	stride 1 time = 0.09ms stride 2 time = 0.09ms stride 3 time = 0.10ms	stride 1 time = 0.19ms stride 2 time = 0.17ms stride 3 time = 0.30ms	stride 1 time = 0.24ms stride 2 time = 0.004ms stride 3 time = 0.002ms
472 * 524	stride 1 time = 0.36ms stride 2 time = 0.54ms stride 3 time = 0.63ms	stride 1 time = 1.06ms stride 2 time = 1.07ms stride 3 time = 2.16ms	stride 1 time = 1.56ms stride 2 time = 0.009ms stride 3 time = 0.008ms

从表中可以看出，使用共享内存可以提升程序性能。

im2col卷积



实验中，不同的Kernel分别提取图像的不同特征。

基础版本核函数

图像大小\线程块大小	8*8	16*16	32*32
176 * 193	stride 1 time = 243.11ms stride 2 time = 0.75ms stride 3 time = 0.73ms	stride 1 time = 240.92ms stride 2 time = 0.78ms stride 3 time = 0.68ms	stride 1 time = 244.80ms stride 2 time = 0.80ms stride 3 time = 0.73ms
472 * 524	stride 1 time = 238.92ms stride 2 time = 1.70ms stride 3 time = 1.13ms	stride 1 time = 237.72ms stride 2 time = 2.46ms stride 3 time = 1.41ms	stride 1 time = 246.86ms stride 2 time = 2.84ms stride 3 time = 1.49ms

从图中可以看出，使用im2col方法时，stride = 1的运行时间远大于stride = 2 和 stride = 3.

图片大小对stride = 2 和 stride = 3的运行时间影响较大，图片越大运行时间越长。

线程块大小对性能的影响不大。

修改任务划分方式（每个线程输出四个像素）

图像大小\线程块大小	8*8	16*16	32*32
176 * 193	stride 1 time = 251.63ms stride 2 time = 0.92ms stride 3 time = 0.80ms	stride 1 time = 255.95ms stride 2 time = 0.96ms stride 3 time = 0.87ms	stride 1 time = 247.29ms stride 2 time = 0.99ms stride 3 time = 1.03ms
472 * 524	stride 1 time = 241.41ms stride 2 time = 3.37ms stride 3 time = 1.32ms	stride 1 time = 241.95ms stride 2 time = 3.38ms stride 3 time = 1.22ms	stride 1 time = 250.38ms stride 2 time = 3.33ms stride 3 time = 1.32ms

从表中可以看出，增加每个线程处理的像素数量，会降低程序性能，但降低的幅度没有直接卷积方法大。

共享内存

图像大小\线程块大小	8*8	16*16	32*32
176 * 193	stride 1 time = 239.98ms stride 2 time = 0.78ms stride 3 time = 0.72ms	stride 1 time = 239.81ms stride 2 time = 0.78ms stride 3 time = 0.61ms	stride 1 time = 233.92ms stride 2 time = 0.66ms stride 3 time = 0.64ms
472 * 524	stride 1 time = 767.31ms stride 2 time = 0.98ms stride 3 time = 0.82ms	stride 1 time = 489.40ms stride 2 time = 71.25ms stride 3 time = 1.12ms	stride 1 time = 505.48ms stride 2 time = 1.35ms stride 3 time = 1.33ms

从表中可以看出，使用共享内存对程序性能的影响不大。

cuDNN卷积

图像大小	
176 * 193	stride 1 time = 0.25ms stride 2 time = 0.19ms stride 3 time = 0.12ms
472 * 524	stride 1 time = 0.94ms stride 2 time = 0.96ms stride 3 time = 0.47ms

从表中可以看出，cuDNN卷积方法是三种方法里性能最强的。

实验感想

通过本次实验，我学到了CUDA卷积的三种方法，能够更加熟练地运用CUDA写出并程序序。